

Adding [nothrow-]swappable traits, revision 3

- [Revision History](#)
- [Discussion](#)
- [Design Rationale Extension II](#)
- [Resolved Issues](#)
- [Related Issues](#)
- [Proposed Resolution](#)
- [Feature-testing Macro](#)
- [Sample Implementation](#)

Revision History

Changes since [P0185R0](#):

- Upon request from LWG further non-member `swap` overloads have been constrained via `std::is_swappable<>` to ensure that the overload set of binary non-member `swap` is consistently constrained.

Changes since [N4511](#):

- A new C++ library issue occurred ([LWG 2554](#)), that would be resolved by the resolution in this paper.
- A new library fundamental issue occurred ([LWG 2561](#)), that could be resolved by the components that are suggested in the resolution in this paper.
- Following the spirit of [P0006R0](#), which had been accepted during the Kona 2015 meeting, this revision also proposes corresponding type trait variable templates.
- An [additional section](#) is provided that explains why the usage of the trait `is_nothrow_swappable<T>::value` in the exception-specification of the array `swap` overload is not only useful but actually indeed important.
- An [example implementation section](#) has been added.

Changes since [N4426](#):

- During a straw pool vote in Lenexa of the Library Working Group there was a strong consensus for accepting variant [III Extended + Constrained swap](#):

none of this	I minimalistic	II extended	III extended + constrained swap
0	0	0	9

Therefore this revision provides now a single wording proposal instead of pointing out possible variants.

- An [additional section](#) is provided that explains why the positive result of `std::is_swappable<std::string&&>` is not only useful but actually indeed important.
- The *Proposed Resolution* has been extended by a missing edit of the header `<utility>` synopsis.
- A new [Related Issues](#) section has been added that points out a similar problem in the C++ Extensions for Library Fundamentals working paper.

Discussion

This proposal suggests to add new type traits `std::is_swappable<T>`, `std::is_swappable_with<T, U>`, `std::is_nothrow_swappable<T>`, and `std::is_nothrow_swappable_with<T, U>` as well as their corresponding variable template counterparts to the header `<type_traits>` to resolve the existing library issue [LWG 2456](#) involving broken `noexcept`-specifications of several member `swap` functions of a number of library components.

In addition to that, this paper suggests to specify the two `swap` templates from header `<utility>` as *constrained* templates.

This paper revision does not repeat the section [Design Rationale Extension](#) of it's predecessor (revision 1) [N4511](#) nor does it repeat contents of it's pre-predecessor [N4426](#).

Please refer to the previous papers regarding general background and (extended) rationale.

Additional note:

As part of a recent [libstdc++ fix](#) the two traits `is_swappable` and `is_nothrow_swappable` had been implemented in `libstdc++` (in the implementors namespace) and the two free `std::swap` templates have been successfully constrained according to the specification suggested by this proposal.

Design Rationale Extension II

In the prevision revision of this paper, the drafting notes in bullet 2 of [the proposed resolution](#) only *recommended* to replace the current expression

```
noexcept(swap(*a, *b))
```

by the *seemingly* equivalent form

```
is_nothrow_swappable<T>::value
```

within the existing `noexcept`-specification of the `swap` overload for arrays:

```
template <class T, size_t N> void swap(T (&a)[N], T (&b)[N]) noexcept(noexcept(swap(*a, *b)));
```

pointing out that this would be more or less only an implementation detail.

With the appearance of a more recent library issue ([LWG 2554](#)), new light had been shed on this matter and the author of this paper now considers the usage of the type trait expression `is_nothrow_swappable<T>::value` as an *essential* part of the proposed wording.

Due to the lookup rules of declarations, the expression `swap(*a, *b)` is part of a yet not completed function declaration, therefore at this point the compiler only sees the first (non-array) `swap` overload whose own exception-specification is determined by the expression

```
is_nothrow_move_constructible<T>::value &&  
is_nothrow_move_assignable<T>::value
```

But since any array type `T` neither meets `is_move_constructible<T>` nor `is_move_assignable<T>`, the `noexcept`-specification *always* evaluates to false.

If instead the type traits expression `is_nothrow_swappable<T>::value` is used, the lookup includes the array `swap` as well.

Resolved Issues

If the proposed resolution would be accepted, the following library issues will be resolved:

Number	Description
2456	Incorrect exception specifications for 'swap' throughout library
2554	Swapping multidimensional arrays is never <code>noexcept</code>

Related Issues

The current [C++ Extensions for Library Fundamentals](#) is affected by very same problems compared to those pointed out in [LWG 2456](#). For example, the specification of `std::experimental::optional`'s member `swap` in 5.3.4 [optional.object.swap] is:

```
void swap(optional<T>& rhs) noexcept(see below);
```

with:

Remarks: The expression inside `noexcept` is equivalent to:

```
is_nothrow_move_constructible_v<T> && noexcept(swap(declval<T>(), declval<T>()))
```

Another affected example is `propagate_const`'s member `swap` function.

Both these cases are now taken care of by the new library fundamentals issue [LWG 2561](#).

Albeit solvable by the components suggested by this proposal, this proposal's suggested resolution does *not* provide concrete wording for [LWG 2561](#)! The author of this paper would prefer to get the principal idea accepted

before attempting to apply it to other working papers.

Proposed Resolution

At some places below, additional markup of the form

[Drafting notes: whatever — end drafting notes]

is provided, which is *not* part of the normative wording, but is solely shown to provide additional information to the reader about the rationale of the concrete wording.

[Drafting notes: This paper generally uses class template traits instead of variable trait definitions to express derived normative specifications for the following reasons:

1. The author plans to write a different proposal that suggests to introduce the new trait predicate combinator `std::conjunction/disjunction` in all specification places where multiple compile-time conditions occur to require short circuit evaluation wherever possible, but the non-lazy evaluation of variable templates within n-ary expression would defeat that purpose, while class templates based traits are (currently) more suitable to realize this.
2. For technical reasons the author considers class template traits as the more fundamental ("primary") form to define a trait. If the working draft editor considers the introduction of variable templates as editorially preferable, he is anyway free to rephrase the proposed wording changes.
3. There are currently still some relevant core language issues open in regard to variable templates (such as CWG 1729 or CWG 1845), and the author therefore didn't want to setup a Library specification that might depend on possibly surprising outcomes of these issues.

— end drafting notes]

The proposed wording changes refers to the Standard C++ working draft [N4567](#).

1. Change header `<utility>` synopsis, 20.2 [utility] p2, as indicated:

```
[...]
// 20.2.2, swap:
template<class T> void swap(T& a, T& b) noexcept(see below);
template<class T, size_t N> void swap(T (&a)[N], T (&b)[N]) noexcept(is_nothrow_swappable<T>::value noexcept(swap(*a, *b)));
[...]
```

2. Change 20.2.2 [utility.swap] as indicated:

[Drafting notes:

1. The *Requires* elements have been left intentionally, based on the assumption that the additional semantic constraints implied by the `MoveConstructible` and `MoveAssignable` are important to keep.
2. The seemingly recursive relation between the `is_[nothrow_]swappable` trait definitions and especially of the second `swap` declaration is resolvable for a concrete implementation by a suitable sequence of (non-defining) declarations of type trait templates and function templates (see the [example implementation](#)).

— end drafting notes]

```
template<class T> void swap(T& a, T& b) noexcept(see below);
```

-1- Remarks: This function shall not participate in overload resolution unless `is_move_constructible<T>::value` is true and `is_move_assignable<T>::value` is true. The expression inside `noexcept` is equivalent to:

```
is_nothrow_move_constructible<T>::value &&
is_nothrow_move_assignable<T>::value
```

-2- Requires: Type `T` shall be `MoveConstructible` (Table 20) and `MoveAssignable` (Table 22).

-3- Effects: [...]

```
template<class T, size_t N>
void swap(T (&a)[N], T (&b)[N]) noexcept(is_nothrow_swappable<T>::value noexcept(swap(*a, *b)));
```

-?- Remarks: This function shall not participate in overload resolution unless `is_swappable<T>::value` is true.

-4- *Requires*: `a[i]` shall be swappable with (17.6.3.2) `b[i]` for all `i` in the range `[0, N)`.

-5- *Effects*: `swap_ranges(a, a + N, b)`

3. Change 20.3.2 [pairs.pair] as indicated:

```
void swap(pair& p) noexcept(see below);
```

-28- *Remarks*: The expression inside `noexcept` is equivalent to:

`is_nothrow_swappable<first_type>::value`noexcept(`swap(first, p.first)`) &&
`is_nothrow_swappable<second_type>::value`noexcept(`swap(second, p.second)`)

[...]

4. Change 20.3.3 [pairs.spec] as indicated:

```
template<class T1, class T2> void swap(pair<T1, T2>& x, pair<T1, T2>& y)  
noexcept(noexcept(x.swap(y)));
```

-7- *Effects*: `x.swap(y)`

-?- Remarks: This function shall not participate in overload resolution unless `is_swappable<T1>::value` is true and `is_swappable<T2>::value` is true.

5. Change 20.4.2.3 [tuple.swap] as indicated:

```
void swap(tuple& rhs) noexcept(see below);
```

-1- *Remarks*: The expression inside `noexcept` is equivalent to the logical AND of the following expressions:

`is_nothrow_swappable<Ti>::value`noexcept(`swap(decval<Ti>>(), decval<Ti>>())`)

where T_i is the i^{th} type in `Types`.

6. Change 20.4.2.9 [tuple.special] as indicated:

```
template <class... Types>  
void swap(tuple<Types...>& x, tuple<Types...>& y) noexcept(see below);
```

-1- *Remarks*: This function shall not participate in overload resolution unless `is_swappable<Ti>::value` is true for all i , where $0 \leq i$ and $i < \text{sizeof}...(Types)$. The expression inside `noexcept` is equivalent to:

`noexcept(x.swap(y))`

-2- *Effects*: `x.swap(y)`

7. Change 20.8.1.5 [unique.ptr.special] as indicated:

[Drafting notes: The general requirements of the deleter don't imply *swappable*, see 20.8.1.2.5 [unique.ptr.single.modifiers] p6, but this is guaranteed to hold for pointer due to `NullablePointer` requirements. — end drafting notes]

```
template <class T, class D> void swap(unique_ptr<T, D>& x, unique_ptr<T, D>& y) noexcept;
```

-?- Remarks: This function shall not participate in overload resolution unless `is_swappable<D>::value` is true.

-1- *Effects*: Calls `x.swap(y)`.

8. Change 20.10.2 [meta.type.synop], header `<type_traits>` synopsis, as indicated:

```
namespace std {  
[...]  
// 20.10.4.3, type properties:  
[...]  
template <class T> struct is_move_assignable;  
  
template <class T, class U> struct is_swappable_with;  
template <class T> struct is_swappable;  
  
template <class T> struct is_destructible;  
[...]  
template <class T> struct is_nothrow_move_assignable;  
  
template <class T, class U> struct is_nothrow_swappable_with;
```

```

template <class T> struct is_nothrow_swappable;

template <class T> struct is_nothrow_destructible;
[...]
```

// 20.10.4.3, type properties

```

template <class T> constexpr bool is_const_v
= is_const<T>::value;
[...]
```

```

template <class T> constexpr bool is_move_assignable_v
= is_move_assignable<T>::value;
template <class T, class U> constexpr bool is_swappable_with_v
= is_swappable_with<T, U>::value;
template <class T> constexpr bool is_swappable_v
= is_swappable<T>::value;
template <class T> constexpr bool is_destructible_v
= is_destructible<T>::value;
[...]
```

```

template <class T> constexpr bool is_nothrow_move_assignable_v
= is_nothrow_move_assignable<T>::value;
template <class T, class U> constexpr bool is_nothrow_swappable_with_v
= is_nothrow_swappable_with<T, U>::value;
template <class T> constexpr bool is_nothrow_swappable_v
= is_nothrow_swappable<T>::value;
template <class T> constexpr bool is_nothrow_destructible_v
= is_nothrow_destructible<T>::value;
[...]
```

9. Change 20.10.4.3 [meta.unary.prop], Table 49 — "Type property predicates", as indicated:

[Drafting notes:

1. The term *referenceable type*, referred to below, is defined in 17.3.19 [defns.referenceable].
2. The specification below allows, but does not require, that an implementation defines the traits `is_swappable` and `is_nothrow_swappable`, respectively, in terms of the implementation details of the more general traits `is_swappable_with` and `is_nothrow_swappable_with`, respectively.

— end drafting notes]

Table 49 — Type property predicates

Template	Condition	Preconditions
...		
<pre> template <class T, class U> struct is_swappable_with;</pre>	The expressions <code>swap(declval<T>(), declval<U>())</code> and <code>swap(declval<U>(), declval<T>())</code> are each well-formed when treated as an unevaluated operand (Clause 5) in an overload-resolution context for swappable values (17.6.3.2 [swappable.requirements]). Access checking is performed as if in a context unrelated to τ and u. Only the validity of the immediate context of the <code>swap</code> expressions is considered. [Note: The compilation of the expressions can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the "immediate context" and can result in the program being ill-formed. — end note]	τ and u shall be complete types, (possibly cv-qualified) void, or arrays of unknown bound.
<pre> template <class T> struct is_swappable;</pre>	For a referenceable type τ, the same result as <code>is_swappable_with<T&, T&>::value</code>, otherwise false.	τ shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.
...		
		τ and u shall be complete

template <class T, class U> struct is_nothrow_swappable_with;	is swappable with<T, U>::value is true and each swap expression of the definition of is_swappable_with<T, U> is known not to throw any exceptions (5.3.7 [expr.unary.noexcept]).	types, (possibly cv-qualified) void, or arrays of unknown bound.
template <class T> struct is_nothrow_swappable;	For a referenceable type T, the same result as is_nothrow_swappable_with<T&, T&>::value, otherwise false.	T shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.
...		

10. Change 23.3.2.1 [array.overview] p3, class template array overview, as indicated:

```
namespace std {
    template <class T, size_t N>
    struct array
    {
        [...]
        void swap(array&) noexcept(is_nothrow_swappable<T>::valuenoexcept(swap(declval<T&>(), declval<T&>())));
        [...]
    };
}
```

11. Change 23.3.2.3 [array.special] as indicated:

[Drafting notes: 23.3.2.8 [array.zero] does specify that the zero-sized array shall have an *unconditional* noexcept(true) exception specification, which implies that there is no reason to constrain it for N == 0. I decided to add the (relaxed) constraint here, because it was much harder to express in 23.3.2.8 [array.zero]. — end drafting notes]

```
template <class T, size_t N>
void swap(array<T, N>& x, array<T, N>& y) noexcept(noexcept(x.swap(y)));
```

~~?- Remarks: This function shall not participate in overload resolution unless N == 0 or is_swappable<T>::value is true.~~

-1- Effects: x.swap(y);

[...]

12. Change 23.3.2.7 [array.swap] before p1 as indicated:

```
void swap(array& y) noexcept(is_nothrow_swappable<T>::valuenoexcept(swap(declval<T&>(), declval<T&>())));
```

13. Change 23.4.4.1 [map.overview] p2, class template map overview, as indicated:

[Drafting notes: According to 23.2.1 [container.requirements.general] p9:

"Lvalues of any Compare, Pred, or Hash types belonging to a and b shall be swappable"

and furthermore:

"If allocator_traits<allocator_type>::propagate_on_container_swap::value is true, then lvalues of type allocator_type shall be swappable"

(There does not exist a fundamental Swappable requirement of Allocators, see 17.6.3.5 [allocator.requirements] p4)

The latter can be transformed into the following conditional constraint:

```
!allocator_traits<allocator_type>::propagate_on_container_swap::value || is_swappable<allocator_type>::value
```

It is currently unclear from the specification, whether the above quoted requirements are considered as *general* type requirements of Compare, Pred, Hash, and allocator_type or whether they are only imposed for the instantiated swap functions.

In the latter case it would be consistent to make these swap functions constrained functions, in the former case not necessarily so.

This proposal decided to not add such constraints *now*, but to defer this suggestion to a future paper.

— end drafting notes]

```

namespace std {
    template <class Key, class T, class Compare = less<Key>,
              class Allocator = allocator<pair<const Key, T> > >
    class map
    {
        [...]
        void swap(map&)
            noexcept(allocator_traits<Allocator>::is_always_equal::value &&
                    is_nothrow_swappable<Compare>::valuenoexcept(swap(declval<Compare&>(),declval<Compare&>())));
        [...]
    };
}

```

14. Change 23.4.5.1 [multimap.overview] p2, class template multimap overview, as indicated:

```

namespace std {
    template <class Key, class T, class Compare = less<Key>,
              class Allocator = allocator<pair<const Key, T> > >
    class multimap
    {
        [...]
        void swap(multimap&)
            noexcept(allocator_traits<Allocator>::is_always_equal::value &&
                    is_nothrow_swappable<Compare>::valuenoexcept(swap(declval<Compare&>(),declval<Compare&>())));
        [...]
    };
}

```

15. Change 23.4.6.1 [set.overview] p2, class template set overview, as indicated:

```

namespace std {
    template <class Key, class Compare = less<Key>,
              class Allocator = allocator<Key> >
    class set
    {
        [...]
        void swap(set&)
            noexcept(allocator_traits<Allocator>::is_always_equal::value &&
                    is_nothrow_swappable<Compare>::valuenoexcept(swap(declval<Compare&>(),declval<Compare&>())));
        [...]
    };
}

```

16. Change 23.4.7.1 [multiset.overview] p2, class template multiset overview, as indicated:

```

namespace std {
    template <class Key, class Compare = less<Key>,
              class Allocator = allocator<Key> >
    class multiset
    {
        [...]
        void swap(multiset&)
            noexcept(allocator_traits<Allocator>::is_always_equal::value &&
                    is_nothrow_swappable<Compare>::valuenoexcept(swap(declval<Compare&>(),declval<Compare&>())));
        [...]
    };
}

```

17. Change 23.5.4.1 [unord.map.overview] p3, class template unordered_map overview, as indicated:

```

namespace std {
    template <class Key,
              class T,
              class Hash = hash<Key>,
              class Pred = std::equal_to<Key>,
              class Allocator = std::allocator<std::pair<const Key, T> > >
    class unordered_map
    {
        [...]
        void swap(unordered_map&)
            noexcept(allocator_traits<Allocator>::is_always_equal::value &&
                    is_nothrow_swappable<Hash>::valuenoexcept(swap(declval<Hash&>(),declval<Hash&>())) &&
                    is_nothrow_swappable<Pred>::valuenoexcept(swap(declval<Pred&>(),declval<Pred&>())));
        [...]
    };
}

```

18. Change 23.5.5.1 [unord.multimap.overview] p3, class template unordered_multimap overview, as indicated:

```

namespace std {
    template <class Key,

```

```

        class T,
        class Hash = hash<Key>,
        class Pred = std::equal_to<Key>,
        class Allocator = std::allocator<std::pair<const Key, T> > >
class unordered_multimap
{
    [...]
    void swap(unordered_multimap&)
        noexcept(allocator_traits<Allocator>::is_always_equal::value &&
        is_nothrow_swappable<Hash>::value noexcept(swap(declval<Hash>()), declval<Hash>())) &&
        is_nothrow_swappable<Pred>::value noexcept(swap(declval<Pred>()), declval<Pred>()));
    [...]
};
}

```

19. Change 23.5.6.1 [unord.set.overview] p3, class template unordered_set overview, as indicated:

```

namespace std {
    template <class Key,
        class Hash = hash<Key>,
        class Pred = std::equal_to<Key>,
        class Allocator = std::allocator<Key> >
class unordered_set
{
    [...]
    void swap(unordered_set&)
        noexcept(allocator_traits<Allocator>::is_always_equal::value &&
        is_nothrow_swappable<Hash>::value noexcept(swap(declval<Hash>()), declval<Hash>())) &&
        is_nothrow_swappable<Pred>::value noexcept(swap(declval<Pred>()), declval<Pred>()));
    [...]
};
}

```

20. Change 23.5.7.1 [unord.multiset.overview] p3, class template unordered_multiset overview, as indicated:

```

namespace std {
    template <class Key,
        class Hash = hash<Key>,
        class Pred = std::equal_to<Key>,
        class Allocator = std::allocator<Key> >
class unordered_multiset
{
    [...]
    void swap(unordered_multiset&)
        noexcept(allocator_traits<Allocator>::is_always_equal::value &&
        is_nothrow_swappable<Hash>::value noexcept(swap(declval<Hash>()), declval<Hash>())) &&
        is_nothrow_swappable<Pred>::value noexcept(swap(declval<Pred>()), declval<Pred>()));
    [...]
};
}

```

21. Change 23.6.3.1 [queue.defn] p1, class template queue definition, as indicated:

```

namespace std {
    template <class T, class Container = deque<T> >
class queue
{
    [...]
protected:
    Container c;

public:
    [...]
    void swap(queue& q) noexcept(is_nothrow_swappable<Container>::value noexcept(swap(c, q.c)))
    { using std::swap; swap(c, q.c); }
};
}

```

22. Change 23.6.3.5 [queue.special] as indicated:

```

template <class T, class Container>
void swap(queue<T, Container>& x, queue<T, Container>& y) noexcept(noexcept(x.swap(y)));

```

-?- Remarks: This function shall not participate in overload resolution unless is_swappable<Container>::value is true.

-1- Effects: x.swap(y).

23. Change 23.6.4 [priority.queue] p1, class template priority_queue definition, as indicated:

```

namespace std {
    template <class T, class Container = vector<T>,

```

```

    class Compare = less<typename Container::value_type> >
class priority_queue
{
    [...]
protected:
    Container c;
    Compare comp;

public:
    [...]
    void swap(priority_queue& q) noexcept(
        is_nothrow_swappable<Container>::value && is_nothrow_swappable<Compare>::value
        noexcept(swap(c, q.c)) && noexcept(swap(comp, q.comp)))
    { using std::swap; swap(c, q.c); swap(comp, q.comp); }
};
}

```

24. Change 23.6.4.4 [priority_queue.special] as indicated:

[Drafting notes: The container adaptors are *not* containers, so we play safe here and impose constraints on Compare regardless of whether we want these for real container types — *end drafting notes*]

```

template <class T, class Container, class Compare>
void swap(priority_queue<T, Container, Compare>& x,
    priority_queue<T, Container, Compare>& y) noexcept(noexcept(x.swap(y)));

```

-?- Remarks: This function shall not participate in overload resolution unless `is_swappable<Container>::value` is true and `is_swappable<Compare>::value` is true.

-1- *Effects:* `x.swap(y)`.

25. Change 23.6.5.2 [stack.defn], class template stack definition, as indicated:

```

namespace std {
    template <class T, class Container = deque<T> >
    class stack
    {
        [...]
    protected:
        Container c;

    public:
        [...]
        void swap(stack& s) noexcept(is_nothrow_swappable<Container>::value
            noexcept(swap(c, s.c)))
        { using std::swap; swap(c, s.c); }
    };
}

```

26. Change 23.6.5.6 [stack.special] as indicated:

```

template <class T, class Container>
void swap(stack<T, Container>& x, stack<T, Container>& y) noexcept(noexcept(x.swap(y)));

```

-?- Remarks: This function shall not participate in overload resolution unless `is_swappable<Container>::value` is true.

-1- *Effects:* `x.swap(y)`.

Feature-testing Macro

For the purposes of SG10, this paper recommends the macro name `__cpp_lib_is_swappable`.

Sample Implementation

Example implementation for the four swappable type trait class templates and the two constrained swap overloads.

```

#include <cstddef>      // std::size_t
#include <type_traits>  // std::enable_if, ...
#include <utility>      // std::move, std::declval

namespace xstd {

    template<class T>
    struct is_swappable;

    template<class T>

```

```

struct is_nothrow_swappable;

template<class T, class U>
struct is_swappable_with;

template<class T, class U>
struct is_nothrow_swappable_with;

template<class T>
inline
typename std::enable_if<
    std::is_move_constructible<T>::value &&
    std::is_move_assignable<T>::value
>::type
swap(T& a, T& b) noexcept(std::is_nothrow_move_constructible<T>::value &&
    std::is_nothrow_move_assignable<T>::value);

template<class T, std::size_t N>
inline
typename std::enable_if<
    is_swappable<T>::value
>::type
swap(T (&a)[N], T (&b)[N]) noexcept(is_nothrow_swappable<T>::value);

namespace swappable_details {

using xstd::swap;

struct do_is_swappable
{
    template<class T, class =
        decltype(swap(std::declval<T&>(), std::declval<T&>()))
    >
    static std::true_type test(int);

    template<class>
    static std::false_type test(...);
};

struct do_is_nothrow_swappable
{
    template<class T>
    static auto test(int) -> std::integral_constant<bool,
        noexcept(swap(std::declval<T&>(), std::declval<T&>()))
    >;

    template<class>
    static std::false_type test(...);
};

struct do_is_swappable_with
{
    template<class T, class U
        , class =
            decltype(swap(std::declval<T>(), std::declval<U>()))
        , class =
            decltype(swap(std::declval<U>(), std::declval<T>()))
    >
    static std::true_type test(int);

    template<class, class>
    static std::false_type test(...);
};

struct do_is_nothrow_swappable_with
{
    template<class T, class U>
    static auto test(int) -> std::integral_constant<bool,
        noexcept(swap(std::declval<T>(), std::declval<U>()))
        &&
        noexcept(swap(std::declval<U>(), std::declval<T>()))
    >;

    template<class, class>
    static std::false_type test(...);
};

template<class T>
struct is_swappable_impl : decltype(
    do_is_swappable::test<T>(0)
)
{};

```

```

template<class T>
struct is_nothrow_swappable_impl : decltype(
    do_is_nothrow_swappable::test<T>()
)
{};

template<class T, class U>
struct is_swappable_with_impl : decltype(
    do_is_swappable_with::test<T, U>()
)
{};

// The following specialization is just a QoI optimization and
// not actually required:
template<class T>
struct is_swappable_with_impl<T&, T&> : decltype(
    do_is_swappable::test<T&>()
)
{};

template<class T, class U>
struct is_nothrow_swappable_with_impl : decltype(
    do_is_nothrow_swappable_with::test<T, U>()
)
{};

// The following specialization is just a QoI optimization and
// not actually required:
template<class T>
struct is_nothrow_swappable_with_impl<T&, T&> : decltype(
    do_is_nothrow_swappable::test<T&>()
)
{};

} // swappable_details

template<class T>
struct is_swappable : swappable_details::is_swappable_impl<T>
{
};

template<class T>
struct is_nothrow_swappable : swappable_details::is_nothrow_swappable_impl<T>
{
};

template<class T, class U>
struct is_swappable_with : swappable_details::is_swappable_with_impl<T, U>
{
};

template<class T, class U>
struct is_nothrow_swappable_with : swappable_details::is_nothrow_swappable_with_impl<T, U>
{
};

template<class T>
inline
typename std::enable_if<
    std::is_move_constructible<T>::value &&
    std::is_move_assignable<T>::value
>::type
swap(T& a, T& b) noexcept(std::is_nothrow_move_constructible<T>::value &&
    std::is_nothrow_move_assignable<T>::value)
{
    T tmp = std::move(a);
    a = std::move(b);
    b = std::move(tmp);
}

template<class T, std::size_t N>
inline
typename std::enable_if<
    is_swappable<T>::value
>::type
swap(T (&a)[N], T (&b)[N]) noexcept(is_nothrow_swappable<T>::value)
{
    for (std::size_t i = 0; i != N; ++i)
    {
        using xstd::swap;
        swap(a[i], b[i]);
    }
}

```

```
} // xstd
```